

AISHWARY KANTODE

Pune, India • aishwary.kantode2@gmail.com • +91 99238 67189
github.com/cookieark7 • linkedin.com/in/aishwary-kantode • portfolio.arkexperiment.xyz

Full-stack developer with 4+ years of experience building responsive, accessible frontends and scalable, secure backend systems across multiple business verticals — equally comfortable restructuring and debugging existing codebases while keeping business logic intact.

WORK EXPERIENCE

XpressBees

Pune, India

Software Engineer

2024 – Present

Junior Software Engineer

2023 – 2024

Graduate Trainee

2022 – 2023

- Rolled out **JWT authentication** across a **1,000+** endpoint surface that previously had none — RS256 JWKS verification with multi-issuer production/staging trust and key caching — verified by an automated auth-coverage suite asserting every route rejects unauthenticated requests.
- Designed and implemented **role-based access control (RBAC)** end to end: Keycloak realm and client-scoped roles resolved server-side and mirrored in the client to gate navigation, route access and deep links across **199** report paths, replacing silent failures on unauthorized access with an explicit permission boundary.
- Built the **encryption layer** protecting customer addresses and phone numbers, using a versioned ciphertext envelope that lets encrypted and legacy plaintext records coexist so key rotation and backfill run without downtime — then caught and fixed a static-IV flaw in my own AES-256-GCM implementation during review, before it reached production.
- Traced a recurring **SQL-correctness bug** to malformed `IN`-clause handling repeated across **116** call sites in **83** route handlers; fixed it once with a reusable helper instead of patching each site individually.
- Built two ops-facing portals adopted org-wide — **lost-shipment identification** (real-time tracking by hub location and date range, cutting loss-shipment write-offs by **97%**) and a **raise-dispute** portal for clients and vendors with image-capture damage identification (cutting dispute resolution time by **95%**).
- Migrated **180+** Redshift stored procedures into version-controlled parameterized SQL, cutting database round-trips per request from **three to one** and moving business-critical query logic out of the database and into reviewable application code.
- Designed and shipped a **Redis caching layer** across **180+** endpoints — URL-keyed entries through a write-through `res.json` wrapper needing zero controller changes — engineered to degrade open so a Redis outage falls back to direct DB reads instead of failing requests, with event-driven invalidation scoped as the next iteration to tighten staleness beyond the current TTL-based expiry.

PROJECTS

Open Source — SQLMesh

[PR #6072](#) • [PR #6007](#) • [repo](#)

Two merged fixes to **SQLMesh** (3.3k-star open-source Python data-transformation framework, a dbt alternative): rewrote integration-test IDs to use secrets-based random suffixes after parallel runs collided on the same schema across Hive/Iceberg backends, staying under Postgres's 63-character identifier limit; and fixed a ClickHouse adapter bug emitting table comments inline inside `CREATE TABLE ... AS SELECT`, which ClickHouse rejects, by repointing it at the existing `ALTER TABLE` path.

Kite Auth SDK — Python

[github](#) • [blog post](#)

Built to kill manual re-logins against Zerodha's Kite Connect: an async **Python SDK** owning the full token lifecycle — browser-automated login (Playwright), token exchange, persistence and automatic re-auth — plus WebSocket tick streaming, retries, a circuit breaker and rate limiting behind a 90% coverage gate — a real-time data source for trading activity. Also fixed a TOTP window-boundary bug in its own auth flow, written up in “Your TOTP code isn't expiring, it's in the wrong window.”

HTTP/1.1 Server — C++

[github](#) • [live](#) • [blog post](#)

Wrote an **HTTP server** on raw sockets in C++, no framework — request parsing, connection handling, keep-alive and routing — persisting to PostgreSQL, behind Nginx with TLS in Docker Compose. A React kanban board rides on top as the live demo. Wrote up the containerization in “Dockerizing a C++ application with multi-stage builds.”

DevVault

[backend](#) • [cli](#) • [live](#)

Personal knowledge vault across four surfaces — Next.js dashboard, Chrome extension with on-device OCR, Node CLI and a ZSH hook — unified over one Express/Prisma/Postgres backend, plus an **MCP server** exposing it to Claude Code. Search runs on **pgvector**-indexed **embeddings** via local Ollama; the ZSH hook mines shell history for automatic tracking.

SKILLS

Languages	JavaScript (ES2021), TypeScript, Python, C++, SQL
Auth & Security	Keycloak (IdP) — OAuth2/OIDC (Auth Code+PKCE, Device Grant) — JWT (RS256, JWKS) — RBAC — AES-256-GCM
Backend	Node.js, Express, FastAPI, Pydantic, Prisma, asyncio, httpx
Frontend	React 18/19, Next.js, TanStack Table/Query/Virtual, Redux Toolkit, MUI, SASS
Infra & Data	AWS (EC2, S3), PostgreSQL, Amazon Redshift, Redis, Nginx, Docker, Git, CI/CD

EDUCATION

B.Tech, Computer Science — Indira College of Engineering & Management, Pune

2022

[freeCodeCamp](#) — JavaScript Algorithms and Data Structures